Decision Transformer for Offline Limit-Order-Book Trading

Côme Genet and Clément Callaert

Abstract. We address high-frequency trading on Limit Order Book (LOB) data as an *offline* reinforcement learning problem. Instead of learning a value function or a policy gradient, we train a Decision Transformer (DT) [1] to predict actions by conditioning on a target return-to-go (RTG), reformulating the RL problem as causal sequence modelling. Training uses a fixed dataset of 6 000 trajectories generated by six diverse behavioural policies on the FI-2010 benchmark [2]. Evaluation on held-out test data shows that all DT variants accumulate positive PnL with near-zero drawdown, and that RTG conditioning gives direct run-time control over the agent's risk/return profile without retraining.

1 Motivation

Why RL? A Limit Order Book (LOB) is the electronic table used by an exchange to match buyers and sellers (see Appendix B): at each tick, the market publishes the best bid and ask prices and their associated volumes across multiple price levels. Trading on LOB data is naturally formulated as a Markov Decision Process (MDP): the agent receives a stream of LOB observations, chooses a discrete position at each step, and earns a reward that depends on subsequent price movements. Classical supervised approaches (e.g. DeepLOB [3]) predict mid-price direction but do not model the sequential consequences of position management, making RL a more appropriate framework.

Why offline RL? Online RL algorithms (e.g. PPO, DQN) require iterative environment interaction. In financial markets this translates to live or simulated execution, which is costly, risky, and hard to reproduce. Offline RL learns entirely from a fixed dataset of historical trajectories, with no further environment interaction. The Decision Transformer [1] sidesteps the distributional-shift issues of offline value-based methods (e.g. CQL) by recasting the problem as supervised sequence modelling: the agent is trained to imitate trajectories conditioned on their observed return, then steered at test time toward high-return behaviour by specifying a target RTG.

2 Environment

Dataset. FI-2010 [2] provides z-score-normalised snapshots of the 10 best bid and ask price and volume levels for 5 Finnish equities over 10 trading days. We use the no-auction, z-score normalised variant, fold 9: days 1–9 for training, day 10 for evaluation.

State space. $s_t \in \mathbb{R}^{41}$: the most recent LOB snapshot (40 features: ask price, ask volume, bid price, bid volume at each of 10 levels) concatenated with the agent's current inventory $p_{t-1} \in \{-1, 0, +1\}$:

$$s_t = \left[x_t^{(1)}, \ldots, x_t^{(40)}, \, p_{t-1}\right].$$

Including the position in the state is essential: the reward depends on what the agent currently holds, not only on the price move.

Action space. $\mathcal{A} = \{0, 1, 2\}$ encoding a *short* position (-1), *flat* (0), or *long* ($+1$). The action specifies the desired inventory; a trade (and its cost) is triggered only when the target differs from the current position.

Reward function. The reward is mark-to-market PnL minus a transaction friction:

$$r_t = p_{t-1}\left(\mathrm{mid}_t - \mathrm{mid}_{t-1}\right) \; - \; c\,|a_t - p_{t-1}|, \tag{1}$$

where $\mathrm{mid}_t = (\mathrm{ask}_{1,t} + \mathrm{bid}_{1,t})/2$ and $c \geq 0$ is the transaction cost per unit of position change. The reward is sparse: the agent only earns when it holds a non-zero position and the market moves in its favour.

Return-to-Go. The RTG at step t is the sum of all future rewards in the episode:

$$\hat{R}_t = \sum_{t'=t}^{T} r_{t'}, \qquad \text{computed backwards: } \hat{R}_t = r_t + \hat{R}_{t+1}, \quad \hat{R}_T = r_T. \tag{2}$$

It is the conditioning signal for the DT: "from this point on, I want to achieve $\hat{R}_t$ more reward."

Episode structure. Training episodes are random 2 000-step windows from the training data. Evaluation runs mean over 32 random test episodes of length 2 000 (zero transaction cost).

3 Agent

3.1 Offline Dataset and Behavioural Policies

The DT learns by imitating trajectories conditioned on their RTG; it therefore requires a dataset that spans a wide range of return quality. We collect $N = 1{,}000$ episodes per policy, 6 000 trajectories in total (≈12M transitions), using the six policies below.

Policy	Description
Oracle	Trades in the direction of the true next mid-price move (upper bound)
Label-momentum	Follows FI-2010 horizon-50 labels: buy if up, sell if down
Imbalance	Long when bid vol > ask vol at level 1, short otherwise
Momentum	Long when mid-price rose over last 20 steps, short otherwise
Mean-reversion	Opposite of label-momentum (intentionally poor)
Random	Uniform random action

Policies of deliberately different quality are critical: without both high- and low-return trajectories, the DT cannot learn the mapping from conditioning return to action. After generation, only the top 50% of trajectories by episode return are retained (`quality_filter_mode: top_quantile`), concentrating training on informative examples while preserving diversity.

3.2 Decision Transformer

The DT frames action prediction as causal language modelling over RL trajectories. A context window of K steps is serialised as an interleaved token sequence of length $3K$:

$$\tau_K = \big(\hat{R}_1,\, s_1,\, a_1,\, \hat{R}_2,\, s_2,\, a_2,\, \ldots,\, \hat{R}_K,\, s_K,\, a_K\big).$$

Embeddings. Each modality is projected to $\mathbb{R}^d$ independently: linear layers for states ($\mathbb{R}^{41} \to \mathbb{R}^d$) and RTGs ($\mathbb{R}^1 \to \mathbb{R}^d$), and an embedding table for discrete actions. A shared learned timestep embedding is added to all three tokens at step t, encoding temporal position within the context. RTGs are standardised before embedding (`standardize_rtg: true`) so that the conditioning signal is scale-invariant across policies.

Causal Transformer backbone. The $3K$-token sequence passes through n_{layers} GPT-2 pre-norm blocks, each comprising masked multi-head self-attention and a two-layer MLP with residual connections and LayerNorm. The causal mask ensures that the prediction of a_t conditions only on $(\hat{R}_{\leq t},\, s_{\leq t},\, a_{<t})$, not on any future token. Only the output at each *state* position feeds the action head:

$$\hat{a}_t = \operatorname{argmax}\big(\mathbf{W}_a\, \mathbf{h}_{s_t} + \mathbf{b}_a\big), \qquad \mathbf{W}_a \in \mathbb{R}^{|\mathcal{A}| \times d}.$$

The model is trained end-to-end with cross-entropy loss between predicted and ground-truth actions.

Hyperparameters. $d = 128$, $n_{\text{heads}} = 4$, $n_{\text{layers}} = 2$, $K = 100$; AdamW with cosine LR decay (lr $= 10^{-4}$, weight decay 10^{-4}); 20 epochs, batch size 2 048.

3.3 Inference Protocol

The agent is initialised with a user-specified target return $\hat{R}^\star$. At each step it feeds the last K context tokens to the DT, executes the argmax action, observes reward r_t, and decrements the RTG: $\hat{R}_{t+1} = \hat{R}_t - r_t$. No gradient updates occur at test time. The target return therefore acts as a run-time dial: varying $\hat{R}^\star$ steers the agent across a spectrum of risk/return profiles from a single trained checkpoint.

4 Theoretical Analysis: From RL to Sequence Modeling

To formally understand the empirical results observed in the subsequent sections, it is necessary to contextualize the Decision Transformer within the broader mathematical framework of Offline Reinforcement Learning and Natural Language Processing.

4.1 Circumventing the Extrapolation Error

In standard value-based RL, the objective is to learn the optimal action-value function $Q^*(s,a)$ via the Bellman optimality operator $\mathcal{B}^*$:

$$\mathcal{B}^* Q(s,a) = \mathcal{R}(s,a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}} \left[\max_{a' \in \mathcal{A}} Q(s',a') \right]$$

In an offline setting, the dataset $\mathcal{D} = \{\tau_i\}$ is fixed and generated by a behavioral policy π_β. The fundamental challenge of offline RL is the *distributional shift* (or extrapolation error). When evaluating the target value, the maximization step $\max_{a'} Q(s',a')$ queries the neural network on actions a' that may fall outside the support of π_β (Out-Of-Distribution actions). Let Q_θ be the current estimate and Q^π the true value function of the induced policy. The extrapolation error ϵ is given by:

$$\epsilon = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} \left[Q_\theta(s,a) - Q^\pi(s,a) \right]$$

This error systematically accumulates over iterative Bellman backups, leading to severe overestimation of OOD actions. The Decision Transformer avoids this theoretical pitfall by entirely abandoning the dynamic programming paradigm in favor of conditional Behavioral Cloning.

4.2 Self-Attention and the Causal Masking

The core mechanism allowing the DT to model RL trajectories is the Multi-Head Self-Attention. Let $X \in \mathbb{R}^{3K \times d}$ be the matrix of sequence embeddings. The queries, keys, and values are obtained via linear projections: $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. The attention is computed as:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M \right) V$$

In the context of the Markov Decision Process, the causal matrix $M \in \mathbb{R}^{3K \times 3K}$ is critical. It strictly enforces the temporal arrow of time, ensuring that the probability of an action a_t is independent of future states and rewards. It is formally defined as:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

The parameters θ of the Transformer are optimized by minimizing the negative log-likelihood over the offline dataset:

$$\mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}} \left[-\sum_{t=1}^{K} \log \mathbb{P}_\theta(a_t \mid \hat{R}_{\leq t}, s_{\leq t}, a_{<t}) \right]$$

4.3 The Fundamental Limitation: The Stitching Problem

While recasting RL as sequence modeling solves the extrapolation error, it inherently lacks the capability of *stitching*. In classical RL, the Bellman equations allow an agent to connect a sub-optimal trajectory $A \to B$ with another sub-optimal trajectory $B \to C$ to discover the optimal path $A \to C$, even if $A \to C$ was never explicitly observed in $\mathcal{D}$.

Because the DT optimizes a purely supervised loss $\mathcal{L}(\theta)$, it strictly models the joint empirical distribution $P(a \mid \hat{R}, s)$ present in $\mathcal{D}$. It cannot infer actions for a desired return $\hat{R}^\star$ if such a return is not supported by the dataset. As we will demonstrate in the ablation studies, providing an OOD conditioning variable $\hat{R}^\star$ forces the network to project the state into an undefined region of the latent space, invariably leading to degenerate behavior (inaction).

5 Results

Protocol. The main configuration uses the transformer architecture with mid-price reward and manual RTG targets $\hat{R}^\star \in \{0, 0.05, 0.1, 0.2, 0.3\}$. The oracle uses ground-truth future labels and is a non-trainable upper bound.

Cumulative PnL. Figure 1 shows the zoomed DT PnL curves (mean over 32 test episodes). All five RTG variants accumulate positive PnL throughout, with $\hat{R}^\star = 0$ reaching the highest terminal value (≈ 0.014). This ordering reflects a dataset coverage effect: the offline dataset contains many flat-heavy trajectories with low but positive returns, so lower target conditioning is better supported by the training distribution. Mean Reversion also accumulates positive PnL, while Momentum degrades steadily, confirming it is a poor strategy on this dataset.

Drawdown. Figure 2 shows the strongest result: all DT variants maintain a near-zero absolute drawdown (< 0.002) across the full episode, while Momentum accumulates a drawdown of ≈ 0.026, more than ten times larger. This confirms that the DT has learned controlled, risk-aware position management.

Risk-adjusted performance. Figure 3 reports the unannualised Sharpe ratio. All DT variants are positive; the best ($\hat{R}^\star = 0$) reaches ≈ 0.03, comparable to Mean Reversion and well above Momentum, the only negative agent.

Training convergence. Training loss decreases from 0.56 to 0.42 over 20 epochs, with accuracy rising from 81.5% to 85.4%, confirming stable convergence (Fig. 4).

Ablations. Two additional configurations were tested. A CNN state encoder (replacing the linear projection with two `Conv1d` layers) and a shaped-reward variant with log-return state representation both used `rtg_source: train_percentiles` for evaluation, which produced negative conditioning values ($\hat{R}^\star \approx -2.4$) because the filtered dataset is dominated by near-zero or negative RTG transitions. Both

models consequently output near-zero PnL across all RTG targets. This highlights a key sensitivity of the DT framework: inference RTGs must lie within the positive support of the training distribution; negative conditioning systematically suppresses trading activity.

6 Conclusion

Reformulating LOB trading as offline sequence modelling via the Decision Transformer yields an agent that accumulates positive PnL on unseen data with near-zero drawdown, outperforming momentum-based heuristics on risk-adjusted return. The RTG conditioning mechanism provides run-time control over the risk/return profile without retraining. A key finding is the sensitivity of the framework to the RTG conditioning distribution: when inference RTGs fall outside the support of the training data, the agent defaults to inaction. Future directions include online fine-tuning, a DeepLOB-style spatial encoder, and careful RTG normalisation to ensure inference values remain in-distribution.

References

1. Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., Mordatch, I.: Decision transformer: Reinforcement learning via sequence modeling. In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
2. Ntakaris, A., Magris, M., Kanniainen, J., Gabbouj, M., Iosifidis, A.: Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. Journal of Forecasting **37**(8), 852–866 (2018)
3. Zhang, Z., Pletikosic, S., Bojchevski, A., Günnemann, S.: DeepLOB: Deep convolutional neural networks for limit order books. IEEE Transactions on Signal Processing **67**(11), 3001–3012 (2019)

Code. Full implementation available at `https://github.com/produit-scalaire/Decision-Transformer-LOB-Trading/`

A Figures

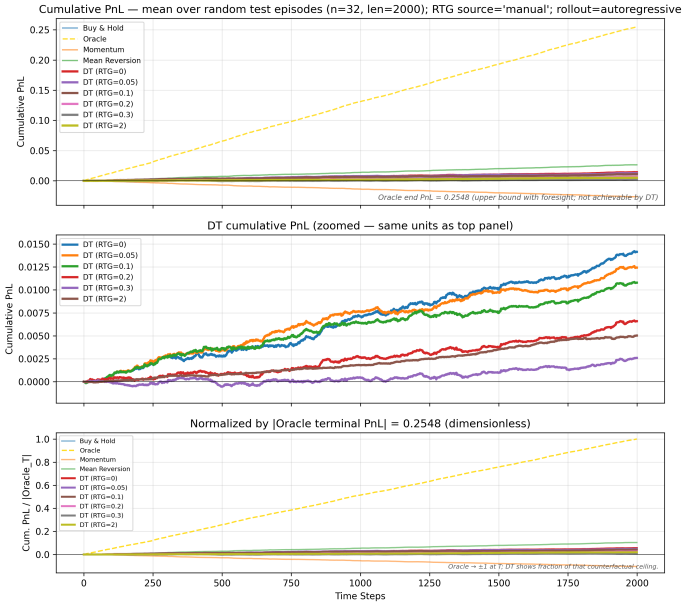

Fig. 1. Cumulative PnL zoomed on DT variants (mean over 32 test episodes). All five RTG targets accumulate positive PnL; $\hat{R}^\star = 0$ reaches the highest terminal value (≈ 0.014).

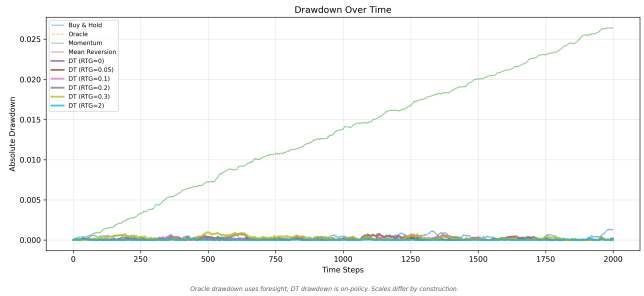

Fig. 2. Absolute drawdown over time. All DT variants remain below 0.002 throughout; Momentum accumulates ≈ 0.026, more than ten times larger.

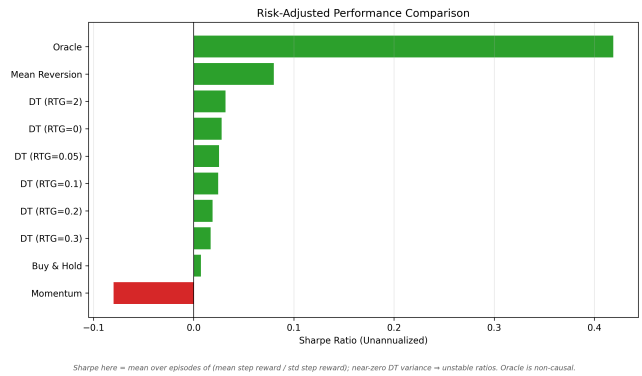

Fig. 3. Unannualised Sharpe ratio. All DT variants are positive; Momentum is the only negative agent.

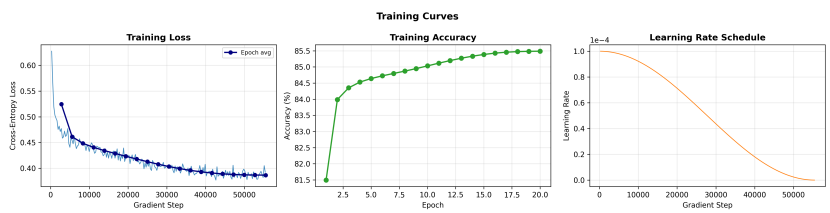

Fig. 4. Training curves: cross-entropy loss (left), action prediction accuracy (centre), cosine LR schedule (right). Loss decreases from 0.56 to 0.42 over 20 epochs; accuracy rises from 81.5% to 85.4%.

B Finance Background: the Limit Order Book

A **Limit Order Book** (LOB) is the electronic table used by an exchange to match buyers and sellers. A *bid* is an intention to buy at a given price; an *ask* is an intention to sell. Orders are organised by price level: level 1 is the best currently available price, level 2 the next best, and so on. At time t, the best prices are $P^a_{1,t}$ (best ask) and $P^b_{1,t}$ (best bid). Three quantities derived from these are central to LOB analysis:

$$\text{mid}_t = \frac{P^a_{1,t} + P^b_{1,t}}{2}, \qquad \text{spread}_t = P^a_{1,t} - P^b_{1,t}, \qquad \text{imbalance}_t = \frac{V^b_{1,t} - V^a_{1,t}}{V^b_{1,t} + V^a_{1,t}},$$

where $V^b_{1,t}$ and $V^a_{1,t}$ are the volumes available at the best bid and ask. One snapshot of the book contains $4 \times 10 = 40$ features (price and volume on each side, at each of the 10 levels).

The LOB is a short-term picture of supply and demand pressure: excess buy volume near the best bid suggests upward price pressure, excess sell volume near the best ask suggests downward pressure, and a narrow spread indicates high liquidity. These microstructural signals motivate using the full 40-feature snapshot as the state representation for our RL agent, rather than the traded price alone.